OPERATIONS / DEVELOPMENT / RECOVERY

OFFLINE IoT DEVELOPMENT KIT

Complete installation, build, editing, customization and troubleshooting handbook

EMQX Enterprise 6.3.1 • Apache Kafka 4.2.2
ThingsBoard CE 4.3.1.6 • Zigbee2MQTT 2.14.2
PostgreSQL 17 • Python BLE gateway

PUBLIC RELEASE / v0.1.0-offline-preview

Windows 10 + openSUSE Leap 16.0 WSL2 + Docker Desktop

Prepared 30 October 2026 • Reference manual, Edition 1

7 ZIP PACKAGES / 24 NESTED ARCHIVES

Published files verified; fresh offline rebuild remains pending

How to use this handbook

This is a practical operating manual for the published seven-ZIP release. It is organized in the order an installer or developer should follow, from connected-PC preparation to offline customization. It distinguishes **verified kit contents** from procedures that still require testing on the second laptop.

Status	What is known
PASS	GitHub prerelease published; seven ZIP archives uploaded; seven ZIP structures checked; 24 nested release archives and 17 required installer/config files found.
PREVIOUSLY TESTED	Source and cached dependency restoration, selected local offline builds, and PostgreSQL backup restore were reported successful during preparation.
NOT YET DEMONSTRATED	A completely fresh, disconnected second-laptop startup and clean rebuild of every component; real Zigbee and BLE hardware integration.

> **Most important requirement**
> The public seven ZIPs do NOT contain the Windows/WSL/Docker installers or the private ThingsBoard PostgreSQL dump. For a functional offline ThingsBoard instance using the current scripts, prepare those separately on the connected computer.

Repository: https://github.com/Developer20003/iot-offline-source-kit

Release: https://github.com/Developer20003/iot-offline-source-kit/releases/tag/v0.1.0-offline-preview

Command labels: Windows PowerShell commands run on the Windows host; **WSL Bash** commands run in the openSUSE terminal; **Docker** commands run in WSL with Docker Desktop integration enabled.

Path convention: `$PKG` is the extracted download package; `$DEV` is the separately restored development workspace. Never restore to the package path or reuse an existing target.

Table of contents

How to use this handbook 2

Table of contents . . . 3

1. Target architecture and scope 6

2. What to download from GitHub 7

What is NOT in the seven ZIPs . . . 7

3. Computer requirements and storage budget 8

4. Prepare all OFFLINE prerequisites while still connected 9

4.1 Inventory the first laptop . . . 9

4.2 Transfer a ready-to-use openSUSE distribution (recommended) . . . 9

4.3 Save installers and hardware drivers . . . 9

4.4 Enable WSL features on the second PC . . . 10

4.5 Docker Desktop on the second PC . . . 10

5. Verify downloads and extract all seven ZIPs 11

5.1 Download and verify top-level ZIPs . . . 11

5.2 Extract into ONE directory . . . 11

5.3 Copy extracted package onto the Linux filesystem . . . 11

6. Verify the installer and original package 12

Source identity: pinned commits . . . 12

7. Restore an independent development workspace 13

Restored directory map . . . 13

Verify restored source checkouts . . . 13

8. Prepare Docker images and Kafka storage 14

Verify resources . . . 14

9. Prepare the PostgreSQL database - essential 15

9.1 Obtain the private database backup on the first laptop . . . 15

9.2 Restore the database on the second laptop . . . 15

10. Start the four core application services 17

User-facing endpoints . . . 17

11. Health checks and a simple offline smoke test 18

11.1 Check containers and TCP listeners . . . 18

11.2 Network-isolated evidence . . . 18

11.3 MQTT check . . . 18

12. Edit sources safely with Git 19
After a change: recommended loop . . . 19
13. Rebuilding EMQX Enterprise 6.3.1 20
13.1 Inspect the build interface . . . 20
13.2 Before restarting EMQX . . . 20
14. Rebuilding Apache Kafka 4.2.2 21
14.1 Inspect and prepare the cache . . . 21
14.2 Example Gradle offline build . . . 21
15. Rebuilding ThingsBoard CE 4.3.1.6 22
15.1 Inspect source and Maven cache . . . 22
15.2 Example offline Maven compile . . . 22
15.3 UI/source customization . . . 22
16. Zigbee2MQTT 2.14.2 - edit, build and run 23
16.1 Adapt configuration for your coordinator . . . 23
16.2 Example offline build and run checks . . . 23
17. Python BLE gateway - edit, install, run 24
17.1 Inspect actual runtime configuration . . . 24
17.2 Install Python packages without internet . . . 24
17.3 Run and validate . . . 24
18. Customize Docker Compose and service settings 25
18.1 Configuration map . . . 25
18.2 ThingsBoard and Kafka are separate by default . . . 25
18.3 Network security and exposure . . . 25
18.4 Kafka listeners and advertised addresses . . . 25
19. Deploy changed builds without losing data 26
20. Backup, maintenance and rollback 27
20.1 Database . . . 27
20.2 Kafka, EMQX and settings . . . 27
20.3 Updating the kit . . . 27
21. Strict offline rebuild acceptance procedure 28
22. Troubleshooting - symptoms and fixes 29
Safe diagnostic capture . . . 29
23. Quick-start cards and command reference 30
23.1 Daily start - after a successful first installation . . . 30
23.2 Daily stop . . . 30

23.3 First-time offline installation sequence 30
23.4 Program-specific edit targets 30

24. Transfer checklist and sign-off 31

Appendix A. Official repository and technical references 32

Appendix B. Licensing, redistribution and version notes 33

1. Target architecture and scope

The kit packages source history, build caches, build resources, prebuilt artifacts, Docker images and scripts. The Docker Compose file orchestrates four services; Zigbee2MQTT and the BLE gateway are packaged but are not currently defined as Compose services.

Part	Role	Packaged / startup
PostgreSQL 17	ThingsBoard database	Docker Compose; requires schema/data
EMQX 6.3.1	MQTT broker + management dashboard	Docker Compose; built runtime restored
Kafka 4.2.2	Event streaming broker	Docker Compose; Kafka KRaft initialized
ThingsBoard 4.3.1.6	IoT application / web UI	Docker Compose; Java boot JAR
Zigbee2MQTT 2.14.2	Zigbee coordinator → MQTT	Source/dist present; separate launch + USB needed
Python BLE gateway	BLE scanning/publish to MQTT	Source + pip wheels present; separate launch + BLE adapter needed

Default network: Docker bridge `iot-backend`. **Ports on the Windows/WSL host:** EMQX MQTT 1883, EMQX dashboard 18083, Kafka 9092, ThingsBoard 8080. Compose publishes these to `127.0.0.1` only. PostgreSQL 5432 stays internal.

Do not assume Kafka is automatically wired to ThingsBoard or EMQX. The delivered ThingsBoard settings specify `TB_QUEUE_TYPE=in-memory`; a Kafka queue or EMQX-to-Kafka bridge would require separate configuration and testing.

2. What to download from GitHub

On an internet-connected computer, open the release page and expand **Assets**. Download the following seven numbered ZIPs and the ZIP SHA256SUMS manifest. Do not confuse this with GitHub's automatically generated "Source code (zip)", which does not contain the 6.6 GB offline kit.

Asset	Approx. size	Contents
01-core-and-sources.zip	982 MB	Four upstream Git bundles, install scripts, portable config, manifest
02-maven-gradle.zip	1.5 GB	Maven and Gradle caches
03-other-dependencies.zip	758 MB	Yarn, pnpm, pip and EMQX caches
04-build-resources.zip	860 MB	EMQX, Kafka and ThingsBoard build resources
05-runtime.zip	640 MB	Compiled runtimes and BLE source
06-docker-images-a.zip	1.5 GB	EMQX builder, Node and Python images
07-docker-images-b.zip	572 MB	Java, Maven and PostgreSQL images
SHA256SUMS	Small	SHA-256 digests for all seven ZIPs

There are two layers of integrity checks: the top-level `SHA256SUMS` verifies the seven ZIPs; the extracted `offline-bundle/RELEASE_SHA256SUMS` verifies the 24 nested application archives.

What is NOT in the seven ZIPs

- Windows installers, WSL runtime/kernel installation package, openSUSE bootstrapping installer, Docker Desktop installer, editor/IDE installers, or offline system-package repositories.
- The private PostgreSQL ThingsBoard backup (.dump), source-device credentials, real USB coordinators and BLE hardware.
- A completed end-to-end clean offline rebuild/startup test report for a second laptop.

3. Computer requirements and storage budget

Category	Recommendation / validation
CPU	64-bit x86 CPU with hardware virtualization enabled (BIOS/UEFI), suitable for packaged linux/amd64 images.
Memory	8 GB is the vendor baseline for Docker WSL2; 16 GB minimum recommended for this multi-service kit; 32 GB helps builds.
Disk	Budget at least 60-80 GB available across Windows + WSL + Docker data for archives, extracted packages, source/caches, loaded images and builds.
Windows	Confirm exact edition, version and build with winver. Current Docker Desktop support requirements must be checked for that edition.
Linux	openSUSE Leap 16.0 WSL2 is the prepared reference environment. Linux build tools must already be installed or transferred offline.
Docker	Working Linux-container Docker Engine and Docker Compose plugin, available inside WSL.
Hardware	A Zigbee coordinator and BLE adapter are optional for core services but mandatory for real hardware validation.

Windows 10 edition compatibility

Docker's current Windows installation guide lists Windows 10 Enterprise, Pro or Education 22H2 build 19045 for its WSL2 backend, with other version requirements. If the offline machine is Windows 10 Home, do not assume that the latest Docker Desktop installer will be supported. Check the exact version or prepare an alternative offline Linux Docker Engine plan.

Check Windows edition and WSL state

```
# Windows PowerShell
winver
systeminfo | findstr /i "OS Name" "OS Version"
wsl --status
wsl --list --verbose
```

4. Prepare all OFFLINE prerequisites while still connected

Perform these preparations on the working, internet-connected PC **before** unplugging or moving the transfer drive. The public kit cannot install Windows components, Docker Desktop, or its Linux toolchain by itself.

4.1 Inventory the first laptop

```
# Windows PowerShell (first laptop)
wsl --list --verbose
wsl --version
# Record Docker Desktop installer/version using the Windows application.

# openSUSE WSL Bash (first laptop)
uname -m
cat /etc/os-release
for c in git python3 tar gzip sha256sum curl docker java javac mvn node pnpm; do
  command -v "$c" || echo "MISSING: $c"
done
docker version
docker compose version
```

Save the output as a text inventory on the transfer drive. Exact Node, Erlang/OTP, JDK, Maven, Gradle wrapper and pnpm versions matter if rebuilding source. The six Docker archives provide runtime/build containers but do not necessarily cover every host compiler, plugin or bootstrap tool.

4.2 Transfer a ready-to-use openSUSE distribution (recommended)

To avoid reinstalling Linux build tools without network access, Microsoft supports exporting the already configured WSL distribution to a TAR and importing it on another Windows machine. The source machine's distro may contain user projects/private settings; inspect it before transferring. Docker Desktop itself is separate and its container images are not fully captured by a distro export.

First Windows laptop: export WSL Linux environment

```
# Windows PowerShell: use your ACTUAL distro name from 'wsl -l -v'
wsl --list --verbose
New-Item -ItemType Directory -Force E:\OfflineTransfer | Out-Null
wsl --shutdown
wsl --export openSUSE-Leap-16.0 E:\OfflineTransfer\openSUSE-Leap16-prepared.tar
```

Replace `openSUSE-Leap-16.0` with the name shown on your system. Transfer the entire exported TAR and verify it. The export may be large and may include project files; it is distinct from the published seven ZIP archives.

4.3 Save installers and hardware drivers

- Obtain the exact Docker Desktop Windows installer suitable for the target Windows edition/version. Test whether it can be installed and started without internet.
- Save the current WSL offline installation/update package if needed; WSL2 optional Windows features and virtualization must work before Linux import.
- If needed, save VS Code or another editor's full offline Windows installer, Git for Windows and any Windows USB passthrough tooling.
- If using Zigbee USB hardware, prepare the Windows-to-WSL USB passthrough tool and drivers separately, then verify the device appears in Linux.
- For a fresh Linux distro rather than WSL export, predownload ALL required RPMs and their dependencies or an offline installation medium; the seven ZIPs do not include such a repository.

4.4 Enable WSL features on the second PC

For Windows PowerShell with administrator privileges, when supported by the target Windows build:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
# REBOOT Windows before continuing.
wsl --status
```

Install any separately prepared offline WSL package if required. Import the TAR only after WSL2 itself works:

```
# Windows PowerShell (second laptop): adapt paths and distro name
New-Item -ItemType Directory -Force D:\WSL\OpenSUSE | Out-Null
wsl --import OpenSUSE-Offline D:\WSL\OpenSUSE `
  E:\OfflineTransfer\openSUSE-Leap16-prepared.tar --version 2
wsl --list --verbose
wsl -d OpenSUSE-Offline
```

Imported WSL distributions commonly start as root by default. If the original `user` account exists, set the default account in `/etc/wsl.conf` with a `[user]` section and `default=user`; restart WSL afterward. Confirm with `id` before restoring files.

4.5 Docker Desktop on the second PC

Install the separately saved compatible Docker Desktop installer, select Linux containers / WSL2 engine, then enable **Settings → Resources → WSL Integration** for the imported distribution. This UI option appears in supported Docker Desktop WSL2 configurations. Restart Docker if necessary.

```
# WSL Bash, second laptop
id
docker info >/dev/null && echo "Docker Engine: READY"
docker compose version
docker images
```

> **No Docker Engine = no offline service startup**
>
> Do not proceed with Docker prep until docker info passes. The kit's preparation/start scripts intentionally stop when Docker is unavailable; they will not install Docker for you.

5. Verify downloads and extract all seven ZIPs

5.1 Download and verify top-level ZIPs

On the connected machine use a browser to save the seven ZIP files and `SHA256SUMS` together in one directory, e.g. `E:\IoT-ZIPs`. Move/copy them to a transfer drive. The final offline computer does not need a GitHub account or GitHub CLI.

Check all seven ZIP downloads

```
# WSL Bash when ZIPs are visible on /mnt/e/IoT-ZIPs
cd /mnt/e/IoT-ZIPs
sha256sum -c SHA256SUMS
# Each of the 7 ZIP names must print ': OK'.
```

If only Windows PowerShell is available, compare every output from `Get-FileHash -Algorithm SHA256` to the corresponding hash in SHA256SUMS. A mismatch means redownload that ZIP on a connected computer.

5.2 Extract into ONE directory

Use Windows Explorer, 7-Zip or PowerShell to extract each numbered ZIP to the **same empty directory**, e.g. `E:\IoT-Extracted`, merging their `offline-bundle` subdirectories. Do NOT extract each ZIP into a different numbered subfolder. Do NOT extract the nested .tar.gz or .bundle files at this stage.

```
# Windows PowerShell; run from folder containing all seven ZIPs
$dest = 'E:\IoT-Extracted'
New-Item -ItemType Directory -Force $dest | Out-Null
Get-ChildItem 'E:\IoT-ZIPs\0*.zip' | Sort-Object Name | ForEach-Object {
    Expand-Archive -LiteralPath $_.FullName -DestinationPath $dest -Force
}
```

In the root of the extracted folder you should find `scripts/`, `offline-bundle/`, README.md and configuration files. Inside `offline-bundle` you should see git, dependencies, build-resources, runtime, docker-images and config.

5.3 Copy extracted package onto the Linux filesystem

For better build performance and file-mode behavior, use the WSL ext4 filesystem under `$HOME/projects` instead of editing directly under `/mnt/e`. Adapt drive letter and destination:

```
# WSL Bash
mkdir -p ~/projects
cp -a /mnt/e/IoT-Extracted ~/projects/iot-offline-package
export PKG="$HOME/projects/iot-offline-package"
cd "$PKG/offline-bundle"
sha256sum -c RELEASE_SHA256SUMS
# Expect 24 archive entries: OK
cd "$PKG"
ls -l offline-bundle/config/compose.yaml scripts/restore-complete-offline-kit.sh
```

Moving archives or restoring into an NTFS-mounted folder can create permission, path and case-sensitivity problems. Prefer Linux ext4 for sources and Docker bind mounts.

6. Verify the installer and original package

The first ZIP already includes 13 installer scripts under `scripts/`. The bootstrap script's `--extract` option refuses to overwrite an existing scripts directory. Therefore use **--verify** for the full seven-ZIP extraction; **--extract** is only for a minimal package that genuinely lacks scripts/.

```
# WSL Bash
export PKG="$HOME/projects/iot-offline-package"
cd "$PKG"
bash offline-bundle/bootstrap-offline.sh --verify
bash scripts/restore-complete-offline-kit.sh --verify
# Expect: COMPLETE OFFLINE PACKAGE VERIFICATION: PASS
```

The full verification script checks source bundles, cached dependencies, build resources, compiled runtimes, portable configuration and Docker image archives. It is a file-integrity check and does **not** start services.

Source identity: pinned commits

Component	Pinned Git commit
EMQX	b0604926195f9372b0068b61f88d409d639a2401
Kafka	c5e942859576203e1b00260070fe6241b64db251
ThingsBoard	c77335f4a524bb9fe0209806672c6ec7aea76141
Zigbee2MQTT	344b6e03b43644c97154a72ca9eae17193c553a2

7. Restore an independent development workspace

The restored development workspace is a new directory that contains working Git source checkouts, caches and runtime files. Do not edit the immutable package archives or restore over the package folder. The master restoration script rejects an existing target; choose a path that does not yet exist.

```
# WSL Bash
export PKG="$HOME/projects/iot-offline-package"
export DEV="$HOME/projects/iot-offline-workspace"
test ! -e "$DEV" || { echo 'Choose a new DEV directory'; exit 1; }
bash "$PKG/scripts/restore-complete-offline-kit.sh" --restore "$DEV"
# Expect: COMPLETE OFFLINE FILE RESTORATION: PASS
```

This runs source Git bundle checkout, cache extraction, EMQX/Kafka/ThingsBoard resource restoration, compiled runtime restoration, Compose/config copy, private credential generation, installer script restoration and a pinned-commit check. It does **not** load Docker images or initialize the database.

Restored directory map

```
iot-offline-workspace/
  compose.yaml
  sources/
    emqx/           # Git code + deps + _build release
    kafka/          # Git code + .gradle resources
    thingsboard/    # Git code + UI node_modules + JAR
    zigbee2mqtt/    # Git code + dist
  caches/           # maven, gradle, yarn, pnpm, pip, emqx
  gateway/          # ble_gateway.py, requirements.txt
  runtime/
    postgres/       # credentials + templates
    emqx/           # env + runtime data
    kafka/          # distribution, config, data
    compose/        # ThingsBoard environment
    zigbee2mqtt/    # configuration template
  logs/
  scripts/
```

Verify restored source checkouts

```
export DEV="$HOME/projects/iot-offline-workspace"
for project in emqx kafka thingsboard zigbee2mqtt; do
  printf '%s: ' "$project"
  git -C "$DEV/sources/$project" rev-parse --short HEAD
  git -C "$DEV/sources/$project" status --short
done
du -sh "$DEV/caches"
```

> **Restoration is not a clean rebuild**
>
> The restored workspace includes some prebuilt artifacts and extracted build directories. Successful restoration demonstrates that files are available; it does not prove fresh compilation of the entire stack.

8. Prepare Docker images and Kafka storage

After Docker Desktop WSL integration works, use the preparation script located in the extracted package. The script rejects container name conflicts and unmanaged Docker resources. On a fresh second laptop, those names should not exist.

```
# WSL Bash
export PKG="$HOME/projects/iot-offline-package"
export DEV="$HOME/projects/iot-offline-workspace"

docker info >/dev/null
bash "$PKG/scripts/prepare-restored-offline-kit.sh" --verify
bash "$PKG/scripts/prepare-restored-offline-kit.sh" --prepare "$DEV"
# Expect: OFFLINE ENVIRONMENT PREPARATION: PASS
```

This step validates Compose, loads all six image archives, creates a labeled bridge network `iot-backend` and a labeled PostgreSQL volume `iot-postgres-data`, and initializes Kafka KRaft storage in `$DEV/runtime/kafka/data` without network access. It does not start the application containers.

Verify resources

```
docker images
docker network inspect iot-backend --format '{{.Name}} {{.Driver}}'
docker volume inspect iot-postgres-data --format '{{.Name}}'
test -s "$DEV/runtime/kafka/data/meta.properties" && echo 'Kafka KRaft: READY'
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" config --quiet
```

If you see “Container iot-postgres already exists” on the source laptop, stop: the scripts purposely prevent collision with the original running stack. Do this on the **fresh** second machine, not over the existing source machine.

9. Prepare the PostgreSQL database - essential

> **Database is a separate transfer item**
>
> The seven public ZIPs do not include the private ThingsBoard database dump. The checked-in startup script requires PostgreSQL 17 data to be initialized, and ThingsBoard needs its SQL schema. Do not assume a blank volume will work. Prepare a private .dump for transfer or implement a separately tested first-run schema installer.

9.1 Obtain the private database backup on the first laptop

Project history reports a successful local 20 MB PostgreSQL dump at `private-backups/postgres/thingsboard-20261030-131859.dump`, restored in an isolated test. If it still exists, verify it rather than creating a new one. This is NOT a public GitHub Release asset.

```
# WSL Bash on original/source laptop
cd ~/projects/iot_offline_source_kit
BACKUP=private-backups/postgres/thingsboard-20261030-131859.dump
test -s "$BACKUP"
docker image inspect postgres:17 >/dev/null
# Check the dump is a valid custom pg_restore archive:
docker run --rm --network none --pull=never -i \
  --entrypoint pg_restore postgres:17 --list < "$BACKUP" >/dev/null
(cd "$(dirname "$BACKUP")" && sha256sum "$(basename "$BACKUP")" > "$(basename "$BACKUP").sha256")
```

If the existing dump is unavailable, create a fresh private backup while the first laptop's PostgreSQL container is healthy. The following uses environment variables already present in the container and never prints the password:

```
# WSL Bash on original laptop (alternative; do NOT use if backup exists)
mkdir -p ~/private-iot-backup
BACKUP="$HOME/private-iot-backup/thingsboard-offline.dump"
docker exec iot-postgres sh -ec \
 'PGPASSWORD="$POSTGRES_PASSWORD" exec pg_dump \
   --format=custom -U "$POSTGRES_USER" -d "$POSTGRES_DB"' > "$BACKUP"
(cd "$(dirname "$BACKUP")" && sha256sum "$(basename "$BACKUP")" > "$(basename "$BACKUP").sha256")
ls -lh "$BACKUP" "$BACKUP.sha256"
```

Transfer the .dump and matching .sha256 by a private USB or trusted file transfer, separate from the seven public ZIPs. The dump can contain credentials, accounts or business data; handle it according to the project's confidentiality requirements.

9.2 Restore the database on the second laptop

Do this only after Step 8 created `iot-postgres-data`. The restore script demands an empty, kit-managed volume and no competing PostgreSQL container. Do not repeat against a populated database volume.

```
# WSL Bash on the second/offline laptop
export PKG="$HOME/projects/iot-offline-package"
export DEV="$HOME/projects/iot-offline-workspace"
BACKUP="$HOME/private-iot-backup/thingsboard-20261030-131859.dump"
# Adapt BACKUP to the actual transferred filename, or thingsboard-offline.dump.

test -s "$BACKUP"
(cd "$(dirname "$BACKUP")" && sha256sum -c "$(basename "$BACKUP").sha256")
bash "$PKG/scripts/restore-offline-postgres.sh" --verify-backup "$BACKUP"
bash "$PKG/scripts/restore-offline-postgres.sh" --restore "$DEV" "$BACKUP"
# Expect: OFFLINE POSTGRESQL DATABASE RESTORATION: PASS
```

Restore logic starts a temporary isolated PostgreSQL 17 container, runs `pg_restore --no-owner --no-acl --exit-on-error`, checks that application tables exist, then removes the temporary container. The database remains in

the persistent Docker volume.

> **No backup available?**
>
> Do not start ThingsBoard yet. The source repository may contain database initialization assets, but the currently published install scripts do not provide a tested clean-empty-database initialization path. That is an outstanding engineering task, not a command to guess.

10. Start the four core application services

Start only after the private PostgreSQL backup has been restored. The startup script checks application files, a valid Compose configuration, locally loaded images, Kafka KRaft metadata, managed Docker resources, and PostgreSQL 17 storage.

```
# WSL Bash, second laptop
export PKG="$HOME/projects/iot-offline-package"
export DEV="$HOME/projects/iot-offline-workspace"

bash "$PKG/scripts/start-offline-services.sh" --verify "$DEV"
bash "$PKG/scripts/start-offline-services.sh" --start "$DEV"
# The script runs: docker compose up -d --no-build --pull never
```

The script should print a Compose service table. A completed start command is not equivalent to passing application health checks. Inspect containers and logs after startup.

```
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" ps
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=80 postgres
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=80 emqx
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=80 kafka
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=80 thingsboard
```

User-facing endpoints

Service	Host endpoint	Notes
EMQX MQTT	127.0.0.1:1883	TCP MQTT; send test MQTT messages with a client
EMQX Dashboard	http://127.0.0.1:18083	Check UI; credentials/config may vary
Kafka	127.0.0.1:9092	Broker listener; internal Docker listener 19092
ThingsBoard	http://127.0.0.1:8080	May need extra startup time and a valid database
PostgreSQL	Internal port 5432	No Windows host port defined in Compose

With Linux containers under Docker Desktop, localhost forwarding behavior depends on host integration; test each address on both the Windows browser and inside WSL. The Compose file intentionally limits host publishing to loopback.

11. Health checks and a simple offline smoke test

11.1 Check containers and TCP listeners

```
# WSL Bash
export DEV="$HOME/projects/iot-offline-workspace"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" ps
curl --max-time 10 -I http://127.0.0.1:18083/ || true
curl --max-time 10 -I http://127.0.0.1:8080/ || true
docker exec iot-postgres sh -ec \
 'pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"'
docker exec iot-kafka sh -lc \
 '/opt/kafka/bin/kafka-topics.sh --bootstrap-server iot-kafka:19092 --list'
```

An HTTP 302/401/login page can still prove a web application is reachable; judge the actual service logs and UI. Kafka commands can fail if the broker has not finished booting; inspect logs and retry after confirming healthy state. Avoid claiming complete IoT integration from port checks alone.

11.2 Network-isolated evidence

For a strict test, disconnect external network access after all prerequisites and archives are installed. Reboot Docker/WSL from the disconnected state and repeat service startup checks with `--pull never`. For container build probes, use `--network none --pull never`. Record which builds succeed and any uncached dependency failures.

11.3 MQTT check

Use an available local MQTT client (or EMQX CLI if packaged) to subscribe to `offline-kit/test`, publish an ordinary test message, and verify the subscriber receives it. Avoid placing authentication data or sensitive telemetry in shell history or public logs.

12. Edit sources safely with Git

All four upstream applications are restored as ordinary Git repositories pinned to known commits. Create your own feature branches before changing code; keep the original package and release assets unchanged.

```
export DEV="$HOME/projects/iot-offline-workspace"
cd "$DEV/sources/thingsboard"
git status --short
git log -1 --oneline
git switch -c custom/my-thingsboard-change
# Edit the required files in VS Code or another offline editor.
git diff --check
git diff
git add path/to/changed/file
git commit -m 'Customize ThingsBoard behavior'
```

Repeat the branch-based method inside `sources/emqx`, `sources/kafka` or `sources/zigbee2mqtt`. Avoid blanket staging generated build directories or secrets. Work from Linux ext4, then use an editor with WSL remote integration only if the extension itself is available offline.

After a change: recommended loop

1. Record the pre-change commit, active branch and relevant configuration.
2. Modify one component at a time and check git diff.
3. Compile using its local/offline toolchain.
4. Run the available unit tests, then restart only the affected service.
5. Perform MQTT/Kafka/HTTP smoke tests; capture logs.
6. Commit source changes and archive any new compiled artifacts for recovery.

13. Rebuilding EMQX Enterprise 6.3.1

The restore places EMQX source in `$DEV/sources/emqx` and extracted Erlang build resources in `deps/` and `_build/`. The installed Docker image is `ghcr.io/emqx/emqx-builder/6.1-8:1.19.1-28.4.1-4-ubuntu24.04`. The Compose runtime executable is `sources/emqx/_build/emqx-enterprise/rel/emqx/bin/emqx`.

13.1 Inspect the build interface

```
export DEV="$HOME/projects/iot-offline-workspace"
cd "$DEV/sources/emqx"
git status --short
ls -ld deps _build
find . -maxdepth 2 -iname '*makefile*' -o -name 'rebar.config*'
grep -nE '^[a-zA-Z0-9_.-]+:' Makefile | head -40
```

Run the build target documented by that exact source commit, using the bundled EMQX builder image and its restored dependencies. We do not assert a single unverified `make` target here because EMQX Enterprise targets and environment settings differ between releases. An earlier local incremental Erlang compilation succeeded, but a fresh clean build from the seven ZIPs is still a separate acceptance test.

13.2 Before restarting EMQX

```
# Snapshot the runtime before testing a new build
cp -a "$DEV/sources/emqx/_build/emqx-enterprise/rel/emqx" \
      "$DEV/sources/emqx/_build/emqx-enterprise/rel/emqx.backup"
# Run the correct offline build target after confirming available dependencies.
# Then validate the resulting release/bin/emqx and restart only EMQX:
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" restart emqx
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=100 emqx
```

If editing the copied release tree or a bridge rule, preserve EMQX data, licensing files and cookie values. Don't erase `runtime/emqx/data` when rebuilding source.

14. Rebuilding Apache Kafka 4.2.2

Kafka source is stored in `$DEV/sources/kafka`, with extracted Gradle resources under `.gradle` and a separate Gradle cache under `$DEV/caches/gradle`. The runtime distribution is `$DEV/runtime/kafka/distribution`.

14.1 Inspect and prepare the cache

```
export DEV="$HOME/projects/iot-offline-workspace"
cd "$DEV/sources/kafka"
ls -l gradlew gradle/wrapper 2>/dev/null || true
find "$DEV/caches/gradle" -maxdepth 2 -type d | head -25
java -version
# Use a compatible JDK and the restored Gradle cache before rebuilding.
```

14.2 Example Gradle offline build

Example only: run this after confirming the cache root layout and wrapper JAR/distribution are present. If the cache archive has an extra nesting level, adjust GRADLE_USER_HOME accordingly.

```
cd "$DEV/sources/kafka"
chmod +x ./gradlew
export GRADLE_USER_HOME="$DEV/caches/gradle"
./gradlew --offline assemble
# Any missing plugin or Gradle distribution must be fetched on a connected PC,
# added to the cache package, and transported back offline.
```

Building Kafka source does **not** automatically replace the deployed Kafka distribution. Produce/test the appropriate Kafka binary distribution and intentionally deploy it to `runtime/kafka/distribution` after backing up the original. Preserve `runtime/kafka/data` and KRaft metadata across application-code changes.

15. Rebuilding ThingsBoard CE 4.3.1.6

The supplied runtime is `$DEV/sources/thingsboard/application/target/thingsboard-4.3.1.6-boot.jar`. Build resources include a Maven cache and `ui-ngx/node_modules`; the Docker Maven image is `maven:3.9.9-eclipse-temurin-17`.

15.1 Inspect source and Maven cache

```
export DEV="$HOME/projects/iot-offline-workspace"
cd "$DEV/sources/thingsboard"
git status --short
ls -l pom.xml application/pom.xml
find "$DEV/caches/maven" -maxdepth 2 -type d | head -25
ls -ld ui-ngx/node_modules
# Confirm local Maven repository is in the expected 'repository' subfolder.
```

15.2 Example offline Maven compile

This is a starting recipe, not yet an end-to-end clean rebuild guarantee. Run it with a locally available Maven installation (or adapt to the packaged Maven Docker image). Some ThingsBoard profiles build the web UI and may require the cached Yarn/Node toolchain.

```
cd "$DEV/sources/thingsboard"
mvn -o \
  -Dmaven.repo.local="$DEV/caches/maven/repository" \
  -DskipTests \
  -pl application -am package
# If artifacts are missing, inspect the failure and add them to a new
# offline cache while internet is available; do not rerun online by accident.
```

Depending on the exact build profile, the produced JAR may require additional packaging. After a successful build, verify the new artifact exists at the precise Compose-mounted path. Restart ThingsBoard only after the JAR is backed up and fully replaced; keep the PostgreSQL volume unchanged.

```
export DEV="$HOME/projects/iot-offline-workspace"
TB_JAR="$DEV/sources/thingsboard/application/target/thingsboard-4.3.1.6-boot.jar"
test -s "$TB_JAR"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" restart thingsboard
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=120 thingsboard
```

15.3 UI/source customization

Frontend source is under `sources/thingsboard/ui-ngx`. Use the lockfiles and build scripts from the pinned repository, the packaged Node/Yarn cache, and strictly offline flags after verifying tool availability. Test UI assets independently; changing frontend files does not alter the running Java JAR until the application is repackaged and redeployed.

16. Zigbee2MQTT 2.14.2 - edit, build and run

Zigbee2MQTT has its own Git checkout at `$DEV/sources/zigbee2mqtt`, a restored `dist/` folder and a local pnpm package store. Its example config is `$DEV/runtime/zigbee2mqtt/configuration.template.yaml`. There is no Zigbee2MQTT service in the published four-service Compose file.

16.1 Adapt configuration for your coordinator

```
export DEV="$HOME/projects/iot-offline-workspace"
mkdir -p "$DEV/runtime/zigbee2mqtt/data"
cp "$DEV/runtime/zigbee2mqtt/configuration.template.yaml" \
   "$DEV/runtime/zigbee2mqtt/data/configuration.yaml"
# Edit 'mqtt.server' and add the actual 'serial.port' and
# 'serial.adapter' for YOUR coordinator.
# Template values: mqtt://iot-emqx:1883, base_topic zigbee2mqtt
```

Use `mqtt://iot-emqx:1883` only from a container joined to the `iot-backend` Docker network. From a WSL host process use the broker's reachable host address, usually `mqtt://127.0.0.1:1883`. Zigbee2MQTT must be able to access the USB coordinator device, often through separately configured Windows/WSL passthrough.

16.2 Example offline build and run checks

```
cd "$DEV/sources/zigbee2mqtt"
node --version
pnpm --version
cat package.json | head -35
# Only when local pnpm/corepack is already installed:
pnpm install --offline --frozen-lockfile --store-dir "$DEV/caches/pnpm"
pnpm run build
# Verify generated dist files; use the repository's documented start script.
```

The user's earlier Zigbee2MQTT local offline pnpm-install/build test passed, but the final USB coordinator and end-to-end Zigbee hardware flow remain untested. Run the service separately or add it to a **new, validated** Compose override with the correct device mapping, mounts and permissions. Do not assume the existing Compose automatically starts it.

17. Python BLE gateway - edit, install, run

The gateway lives in `$DEV/gateway/ble_gateway.py` with `requirements.txt`. Pip wheels were archived into `$DEV/caches/pip`. The Python 3.12 image was packaged; its CLI environment may be used for controlled dependency tests. BLE hardware access from Docker Desktop/WSL requires special care.

17.1 Inspect actual runtime configuration

```
export DEV="$HOME/projects/iot-offline-workspace"
cd "$DEV/gateway"
sed -n '1,180p' requirements.txt
grep -nE 'getenv|os\.environ|argparse|MQTT|BLE|Bleak' ble_gateway.py | head -80
# Identify the exact config keys before supplying MQTT host / device filters.
```

17.2 Install Python packages without internet

```
cd "$DEV/gateway"
python3 -m venv .venv
. .venv/bin/activate
find "$DEV/caches/pip" -maxdepth 2 -type f | head -20
python -m pip install --no-index --find-links "$DEV/caches/pip" \
  -r requirements.txt
python -m pip check
```

If the wheel directory has a different nesting layout, point `--find-links` at the directory holding the wheel files. This installs only from supplied local files and fails if a requirement is absent. If the host lacks Python venv or Bluetooth dependencies, transfer those system packages separately before disconnecting.

17.3 Run and validate

Use the script's own configuration interface (identified above) and the actual Bluetooth adapter. Launch `python ble_gateway.py` from the activated environment only after MQTT/BLE settings are correct. Confirm that a real BLE advertisement results in an MQTT message on the expected topic. The prior gateway/MQTT software tests do not establish actual BLE hardware support on the new laptop.

18. Customize Docker Compose and service settings

The active Compose file is `$DEV/compose.yaml`. The original template is `$PKG/offline-bundle/config/compose.yaml`. The restored file is the intended place for site-specific changes. Save a dated backup and validate Compose after every edit.

```
export DEV="$HOME/projects/iot-offline-workspace"
cp "$DEV/compose.yaml" "$DEV/compose.yaml.before-customization"
# Edit $DEV/compose.yaml using your offline editor.
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" config --quiet
```

18.1 Configuration map

File	Purpose
$DEV/compose.yaml	Core service images, ports, volumes and Docker network
$DEV/runtime/postgres/postgres.env	Database name, user, private password
$DEV/runtime/emqx/emqx.env	EMQX node identity and cookie
$DEV/runtime/compose/thingsboard.env	ThingsBoard SQL connection and queue mode
$DEV/runtime/kafka/config/server.properties	Kafka listeners, KRaft settings, storage
$DEV/runtime/zigbee2mqtt/data/configuration.yaml	Coordinator, MQTT and Zigbee2MQTT settings (create locally)

18.2 ThingsBoard and Kafka are separate by default

The distributed ThingsBoard template uses SQL persistence with an in-memory queue. For Kafka-backed ThingsBoard queueing, you must explicitly update the ThingsBoard queue settings to the correct internal broker address, adjust any required topics/credentials, and test. Do not change `TB_QUEUE_TYPE` alone and assume it is configured.

18.3 Network security and exposure

Local loopback publishing is deliberate. If other LAN computers must connect, first decide which service should be reachable, then review host-port bindings and Windows firewall rules. For a production deployment, supply authentication, access control and TLS; do not expose demo MQTT or web endpoints on a public interface without protection.

18.4 Kafka listeners and advertised addresses

The Kafka server.properties file advertises internal `iot-kafka:19092` and external `127.0.0.1:9092`. Docker-network clients should use the internal listener. A separate LAN client cannot use the loopback advertised address; changing it requires a deliberate end-to-end connectivity test and potentially a new port/firewall plan.

19. Deploy changed builds without losing data

Keep code, compiled releases, configuration and state separate. The PostgreSQL Docker volume, Kafka data directory and EMQX runtime data are not build output and must not be removed during ordinary code changes.

1. Before rebuilding, save Git commit identifiers, relevant config and a current database backup.
2. Stop only the affected service if a binary must be replaced, e.g. docker compose stop thingsboard.
3. Copy the existing binary/release into a dated rollback location on the Linux filesystem.
4. Place the fully compiled and verified replacement into the exact bind-mounted path.
5. Validate Compose (when changed), restart the affected service and inspect logs.
6. Run the application and MQTT/Kafka/SQL smoke tests before marking the change accepted.

```
# Example: restart after replacing ThingsBoard JAR
export DEV="$HOME/projects/iot-offline-workspace"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" stop thingsboard
# Replace only the validated built JAR, preserving its filename.
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" up -d --no-build --pull never thingsboard
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=100 thingsboard
```

Where a component's build requires a fresh Docker image, verify that image is already loaded. Avoid `docker compose build`, `docker pull`, package-manager online modes, or auto-update commands on the isolated computer.

20. Backup, maintenance and rollback

20.1 Database

Take PostgreSQL custom-format dumps while the database is running; verify they can be listed by pg_restore and store copies off-device. The current restore script only accepts an empty kit-managed volume, so a disaster recovery plan must account for safe volume replacement and never blindly restore over live data.

```
# From WSL Bash when iot-postgres is healthy
mkdir -p "$HOME/private-iot-backup"
STAMP="$(date +%Y%m%d-%H%M%S)"
FILE="$HOME/private-iot-backup/thingsboard-$STAMP.dump"
docker exec iot-postgres sh -ec \
 'PGPASSWORD="$POSTGRES_PASSWORD" exec pg_dump \
  -Fc -U "$POSTGRES_USER" -d "$POSTGRES_DB"' > "$FILE"
(cd "$(dirname "$FILE")" && sha256sum "$(basename "$FILE")" > "$(basename "$FILE").sha256")
```

20.2 Kafka, EMQX and settings

- Preserve Kafka $DEV/runtime/kafka/data (including KRaft metadata), and back up topics using application-aware procedures if necessary.
- Preserve $DEV/runtime/emqx/data and all effective EMQX runtime configuration.
- Store $DEV/compose.yaml and the private environment files in an access-controlled backup.
- Keep Git source commits, any local build instructions, tool versions and rebuilt binary hashes with each release.
- Never delete the Docker volume iot-postgres-data as a routine “cleanup” step.

20.3 Updating the kit

The release is frozen at pinned commits. To add newer source versions or missing offline dependencies, do a controlled update on a connected computer: rebuild source bundles and dependency caches, refresh SHA-256 manifests, restore/test in a new directory, then publish a **new** release. Do not silently overwrite a previously verified release or mutate the offline package without regenerating checksums.

21. Strict offline rebuild acceptance procedure

A development kit is truly offline only after proving that its tooling runs without downloads. The recommended sequence is to test one component at a time on a fresh restore while the computer is disconnected from the network, not merely to check for existing prebuilt binaries.

Checkpoint	Pass criteria
Integrity	All seven ZIP hashes match; all 24 nested archive hashes match.
Prerequisites	WSL, Docker, Linux commands and needed build tools work without internet.
Restore	New non-existing DEV path restores four Git repos, six caches, resources and runtimes.
Docker	Six required images load from archives; Compose config validates.
Database	Private dump restores into new PostgreSQL volume; tables present.
Start	Four core services start with --pull never; endpoints and logs pass.
Build	EMQX, Kafka, ThingsBoard and Zigbee2MQTT compiled from source with offline flags.
Edit	Change source, produce artifact, redeploy, verify intended change.
Hardware	Real Zigbee and BLE devices can be used (separate validation).

Use the following offline-safe Compose commands when testing:

```
export DEV="$HOME/projects/iot-offline-workspace"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" config --quiet
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" up -d --no-build --pull never
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" ps
```

Be precise about status

The latest confirmed milestone is published, complete ZIP distribution and structural integrity. Clean second-laptop offline startup and fresh source rebuild are NOT yet confirmed. This acceptance section is a practical checklist to complete, not a claim that every line has passed.

22. Troubleshooting - symptoms and fixes

Symptom	Likely reason / safe next action
gh / GitHub absent	Expected offline. GitHub CLI is only for download/upload on a connected PC; the downloaded kit needs none.
bootstrap --extract refuses	scripts/ already exists after extracting ZIP 01. Use bootstrap --verify.
Missing compose.yaml	Check offline-bundle/config/compose.yaml in package; active compose.yaml appears at DEV root after restoration.
Source restore: target exists	Choose a new, nonexistent DEV directory; never overwrite an existing workspace.
sha256sum mismatch	Corrupt or mismatched asset; re-download/re-copy the named file before continuing.
docker info fails	Install/launch compatible Docker Engine and enable WSL distro integration.
Docker name conflict	Original containers exist. Do not run restore/start over source laptop's iot-* containers.
Unmanaged network/volume	A Docker network or volume has the kit's name but not its management label; inspect ownership before making changes.
Kafka not initialized	Run prepare-restored-offline-kit; verify runtime/kafka/data/meta.properties. Never reformat existing data.
PostgreSQL volume not initialized	Transfer/restore a valid private .dump; current startup requires PostgreSQL PG_VERSION=17 in volume.
Database restore volume not empty	Restore script is intentionally one-shot to an empty volume. Plan recovery carefully; do not delete production data.
ThingsBoard HTTP unavailable	Inspect JAR path, DB restore, datasource settings, and application logs.
Gradle/Maven package missing	Cache incomplete; add specific dependency on connected PC then repack and verify.
Zigbee coordinator missing	Windows USB not forwarded to WSL/container or port/adapter config wrong.
BLE scan finds nothing	Adapter passthrough/permissions or physical devices; container BLE access may be unsupported.
Kafka external clients fail	Advertised listener is loopback; configure reachable hostname and test.

Safe diagnostic capture

```
# WSL Bash
export DEV="$HOME/projects/iot-offline-workspace"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" ps
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" logs --tail=150 > "$HOME/iot-debug.log"
# Review the log and remove credentials/tokens before sharing it.
```

23. Quick-start cards and command reference

23.1 Daily start - after a successful first installation

```
# Windows: start Docker Desktop and open openSUSE WSL.
# WSL Bash:
export DEV="$HOME/projects/iot-offline-workspace"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" up -d --no-build --pull never
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" ps
# EMQX http://127.0.0.1:18083   ThingsBoard http://127.0.0.1:8080
```

23.2 Daily stop

```
export DEV="$HOME/projects/iot-offline-workspace"
docker compose --project-directory "$DEV" -f "$DEV/compose.yaml" down
# 'down' stops/removes service containers; it does NOT remove external
# Postgres volume by default. Never add -v without a deliberate data plan.
```

23.3 First-time offline installation sequence

```
# Assumes: 7 ZIPs extracted, PKG on WSL ext4, Docker ready.
export PKG="$HOME/projects/iot-offline-package"
export DEV="$HOME/projects/iot-offline-workspace"
cd "$PKG/offline-bundle" && sha256sum -c RELEASE_SHA256SUMS
bash "$PKG/offline-bundle/bootstrap-offline.sh" --verify
bash "$PKG/scripts/restore-complete-offline-kit.sh" --verify
bash "$PKG/scripts/restore-complete-offline-kit.sh" --restore "$DEV"
bash "$PKG/scripts/prepare-restored-offline-kit.sh" --prepare "$DEV"
# Required private database file, supplied separately:
BACKUP="$HOME/private-iot-backup/thingsboard-20261030-131859.dump"
# Or use your fresh private dump filename.
bash "$PKG/scripts/restore-offline-postgres.sh" --restore "$DEV" "$BACKUP"
bash "$PKG/scripts/start-offline-services.sh" --start "$DEV"
```

Important: This sequence creates a new DEV directory and an empty managed Docker database volume. Do not rerun the restoration/initialization steps blindly after first installation; use the daily-start commands instead.

23.4 Program-specific edit targets

Software	Edit location	Build/deploy output
EMQX	DEV/sources/emqx/	DEV/sources/emqx/_build/.../rel/emqx/
Kafka	DEV/sources/kafka/	Built distribution → DEV/runtime/kafka/distribution/
ThingsBoard	DEV/sources/thingsboard/	DEV/sources/thingsboard/application/target/...-boot.jar
Zigbee2MQTT	DEV/sources/zigbee2mqtt/	DEV/sources/zigbee2mqtt/dist/
BLE gateway	DEV/gateway/ble_gateway.py	Python source; no prebuilt native executable

24. Transfer checklist and sign-off

Print this section and mark it on the connected computer and again on the offline computer.

- [ ] Download all 7 ZIP packages and matching SHA256SUMS from the published Release.
- [ ] Verify all seven ZIP SHA-256 digests before extraction.
- [ ] Obtain offline Windows/WSL/Docker installers or a prepared WSL distribution export.
- [ ] Prepare required Linux compilers, CLI tools and optional hardware drivers.
- [ ] Transfer the private ThingsBoard PostgreSQL .dump and checksum separately.
- [ ] Extract all seven ZIPs to ONE folder and copy onto the Linux ext4 filesystem.
- [ ] Verify 24 nested archives and complete-installation script checks.
- [ ] Restore sources/caches/runtimes into a NEW development directory.
- [ ] Docker info / Compose work; load six images and initialize Kafka.
- [ ] Restore ThingsBoard database into the empty kit-managed Docker volume.
- [ ] Start four core containers with no image pulls; verify logs and endpoints.
- [ ] Edit/rebuild/roll back each source component without internet.
- [ ] Test real Zigbee coordinator and BLE hardware separately when available.
- [ ] Record test results, build artifacts, hashes, versions and backups.

> **Final result required for true completion**
>
> Only mark the entire environment “fully offline and source-rebuildable” when all needed checkpoints, including clean builds and application start, have been tested without network access on the target laptop. The published seven ZIPs are a verified distribution foundation, not proof of those tests.

Appendix A. Official repository and technical references

The project-specific procedure in this manual was based on public source files in the release repository, including README.md, offline-bundle/bootstrap-offline.sh, config/compose.yaml, source-commits.txt, and the 13 restore/prepare/start scripts. Links below are intended to remain usable as reference when returning to an internet-connected computer.

Public repository
https://github.com/Developer20003/iot-offline-source-kit

Published ZIP release
https://github.com/Developer20003/iot-offline-source-kit/releases/tag/v0.1.0-offline-preview

Master restore script
https://github.com/Developer20003/iot-offline-source-kit/blob/main/scripts/restore-complete-offline-kit.sh

Prepare Docker/Kafka script
https://github.com/Developer20003/iot-offline-source-kit/blob/main/scripts/prepare-restored-offline-kit.sh

PostgreSQL restore script
https://github.com/Developer20003/iot-offline-source-kit/blob/main/scripts/restore-offline-postgres.sh

Start services script
https://github.com/Developer20003/iot-offline-source-kit/blob/main/scripts/start-offline-services.sh

Compose template
https://github.com/Developer20003/iot-offline-source-kit/blob/main/offline-bundle/config/compose.yaml

Microsoft WSL export/import
https://learn.microsoft.com/en-us/windows/wsl/basic-commands

Microsoft custom WSL distro import
https://learn.microsoft.com/en-us/windows/wsl/use-custom-distro

Docker Desktop Windows prerequisites
https://docs.docker.com/desktop/setup/install/windows-install/

Docker Desktop WSL integration
https://docs.docker.com/desktop/features/wsl/

EMQX licensing FAQ
https://www.emqx.com/en/content/license-faq

Appendix B. Licensing, redistribution and version notes

The kit's upstream technologies carry independent third-party licenses. Rebuilding or redistributing their sources, Docker images, JavaScript dependencies and prebuilt executables does not replace those terms. Preserve copyright/license notices and review each component's distribution rights, especially if packaging this kit for public redistribution or commercial hosting.

EMQX Enterprise 6.3.1 is source-available under BSL 1.1 according to the project repository and EMQX's licensing guidance; source availability is not identical to unrestricted OSI-approved open-source use. Docker Desktop may also require a commercial subscription depending on organization size and commercial use. Refer to upstream terms applicable to the actual version and use case.

The current official Docker Desktop requirements are subject to change. Always verify target Windows edition/build and WSL version against the exact Docker installer selected. The prepared developer context is Windows 10 / openSUSE Leap 16.0 WSL2 / Docker Desktop, but installation on a completely different edition or hardware platform may need a separately tested path.

Manual revision: 1.0, issued 30 October 2026. **Release:** v0.1.0-offline-preview. **Scope:** detailed handoff and engineering runbook. **Known gap:** end-to-end fresh offline target machine verification and clean component rebuild remain pending.